

propselect

Electric Propulsion Selection Tool — Solver Reference

Physics, assumptions, equations, and control flow of the coupled motor/propeller/battery solver, for both the takeoff ground-roll and cruise steady-level-flight modes.

1. Overview & Scope

2. System Architecture

3. Takeoff Solver Flow Chart

4. Nomenclature

5. Atmosphere Model

6. Battery Model

7. Motor Model

8. Propeller Model

9. Coupled Operating-Point Solver
10. Multi-Motor Extension

11. Ground-Roll Integration (Takeoff)

12. Cruise (Steady Level Flight)

13. Closed-Form Quick Estimate

14. Candidate Filters

15. Momentum-Theory Cross-Check

16. Assumptions & Limitations

17. Validation

18. Module Reference

1. Overview & Scope

propselect selects and ranks electric motor + propeller combinations for a fixed-wing aircraft against a takeoff-distance requirement and/or a cruise (steady, level-flight) requirement. For each candidate (a motor, a propeller, and a motor count, direct drive only), it solves the coupled motor/propeller electrical-mechanical operating point — across an airspeed sweep for takeoff, or at a single thrust-equals-drag equilibrium for cruise — and scores the result against a set of filters (tip Mach, current, power, efficiency, distance or endurance).

The physics core (`propselect/core/`) has zero GUI dependencies and is fully scriptable headlessly; the GUI and CLI are thin front ends over the same functions documented here.

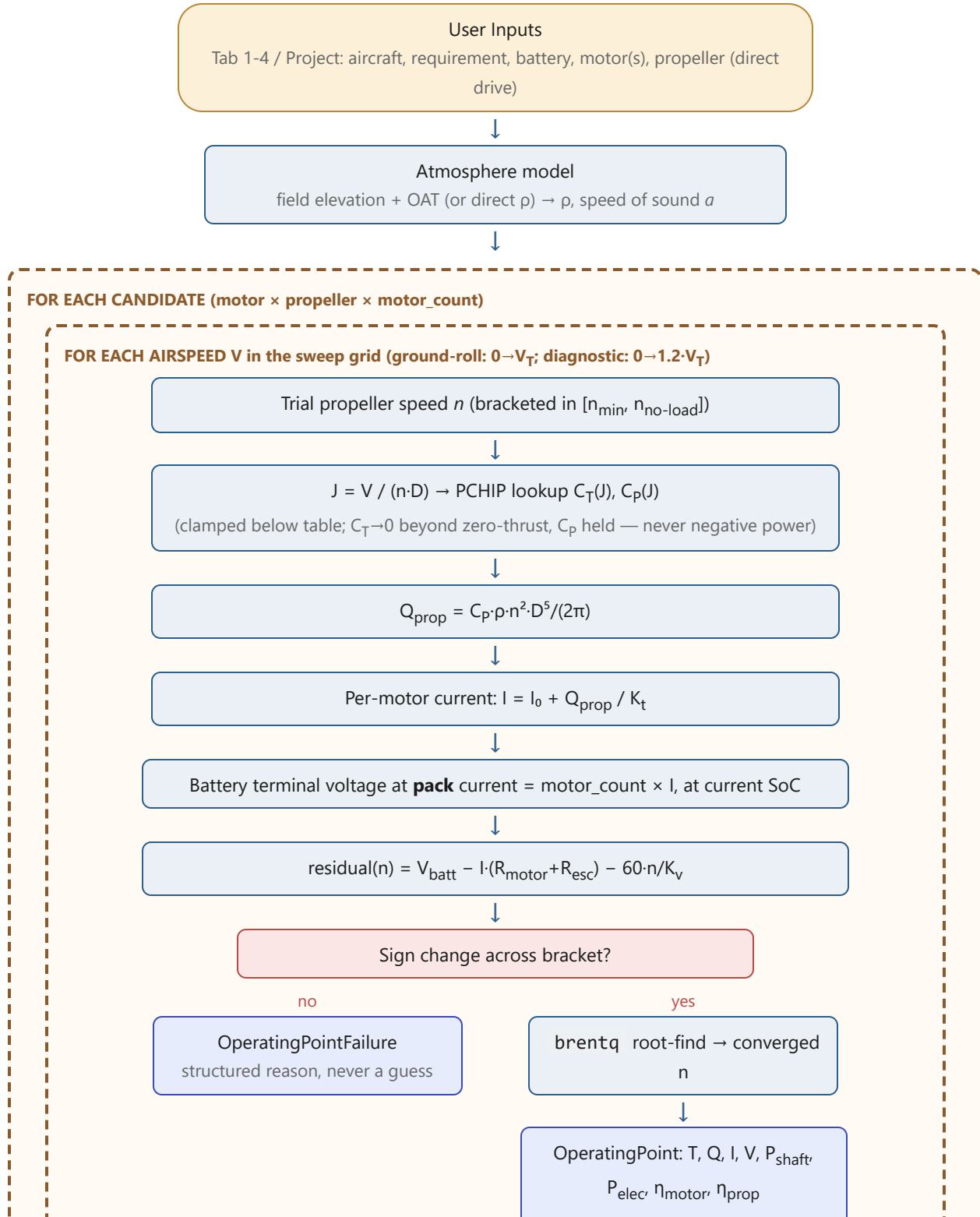
2. System Architecture

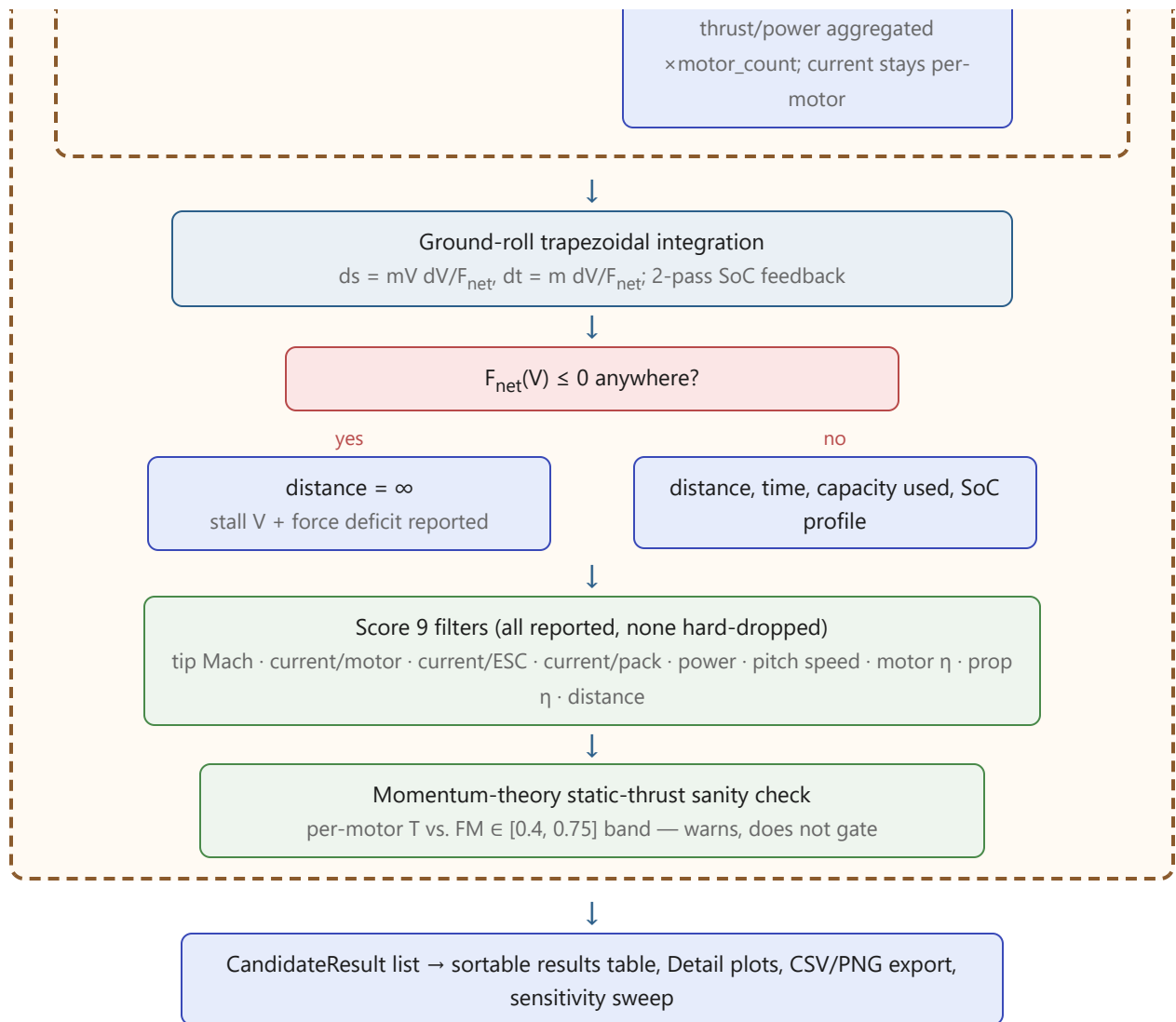
Package	Responsibility
<code>core/atmosphere.py</code>	ISA density, pressure, speed of sound from field elevation/temperature, or direct entry.
<code>core/battery.py</code>	Two selectable pack voltage-sag models (internal resistance, measured curve).
<code>core/motor.py</code>	BLDC motor electrical/torque relations (direct drive).
<code>core/propeller.py</code>	$C_T(J)$, $C_P(J)$ interpolation, thrust/power/torque, momentum-theory check.
<code>core/operating_point.py</code>	The coupled 1-D root-find solver at a single airspeed (full throttle).

<code>core/takeoff.py</code>	Ground-roll trapezoidal integration, SoC feedback, closed-form estimate.
<code>core/cruise.py</code>	Steady level-flight ($T=D$) equilibrium solve, throttle/endurance, cruise filters.
<code>core/candidate.py</code>	Per-candidate takeoff evaluation: runs the ground roll, sweeps diagnostics, scores all filters.
<code>data/loaders.py</code>	APC <code>.dat</code> , UIUC, and CSV propeller data import with unit-consistency validation.
<code>project.py</code>	JSON-serializable input model shared by the GUI and CLI; converts to the core dataclasses above (both takeoff and cruise requirements).
<code>main.py</code>	CLI entry point (<code>propselect cli --mode takeoff cruise both</code>) and GUI launcher (<code>propselect gui</code>).
<code>gui/</code>	PySide6 desktop app: 10 tabs, 3 of which (Cruise, Cruise Detail, Cruise Comparison) mirror the takeoff Sweep/Detail/Comparison tabs for the cruise workflow.

3. Takeoff Solver Flow Chart

See §12 for the cruise solve, which replaces the ground-roll integration below with a single equilibrium root-find and has no diagnostic sweep or SoC-feedback loop.





4. Nomenclature

Symbol	Meaning	Unit
V	airspeed	m/s
n	propeller shaft speed	rev/s
D	propeller diameter	m
J	advance ratio, $V/(nD)$	—
C_T, C_P	thrust / power coefficients	—
ρ	air density	kg/m ³
T, Q, P	thrust, torque, power	N, N·m, W
K_V, K_t	motor velocity / torque constants	RPM/V, N·m/A
I_o	motor no-load current	A
$R_{\text{motor}}, R_{\text{esc}}$	motor winding / ESC resistance (per motor branch)	Ω
$V_{\text{ocv}}, R_{\text{internal}}$	per-cell open-circuit voltage, internal resistance	V, Ω
S, b, AR	wing area, span, aspect ratio (= b^2/S , derived)	m ² , m, —
C_D, C_L	ground-roll drag / lift coefficients	—
μ	rolling friction coefficient	—
m, g	aircraft mass, gravity (9.80665)	kg, m/s ²
F_{net}	net accelerating force	N
s, t	ground-roll distance, time	m, s
FM	figure of merit (actuator disk)	—

5. Atmosphere Model

ISA (troposphere, <11 km) with an optional offset, or direct entry of density and temperature. Source: `core/atmosphere.py`.

$T(h) = T_0 + \Delta ISA - L \cdot h$	(ISA temperature)
$P(h) = P_0 \cdot (1 - L \cdot h / T_0)^{(g_0 / (R \cdot L))}$	(ISA pressure)
$a = \sqrt{\gamma \cdot R \cdot T}$	(speed of sound)
$\rho = P / (R \cdot T)$	(ideal gas density)

Constant	Value	Source
γ (air)	1.4	ratio of specific heats, dry air
R (air)	287.05 J/(kg·K)	specific gas constant, dry air
g_0	9.80665 m/s ²	standard gravity
T_0, P_0	288.15 K, 101325 Pa	ISA sea-level reference
L	0.0065 K/m	ISA tropospheric lapse rate

The "field elevation + OAT" mode uses the ISA pressure profile at the given elevation but the *actual measured* temperature for density (standard density-altitude convention) rather than the ISA-standard temperature at that altitude, since OAT is normally known directly.

6. Battery Model

Model A — internal resistance (default)

$$\begin{aligned}
 V_{\text{ocv,pack}}(\text{SoC}) &= V_{\text{ocv,cell}}(\text{SoC}) \cdot \text{series} \quad [V_{\text{ocv,cell}} \text{ via PCHIP of an OCV table}] \\
 R_{\text{pack}} &= R_{\text{cell}} \cdot \text{series} / \text{parallel} \\
 V_{\text{terminal}}(I) &= V_{\text{ocv,pack}}(\text{SoC}) - I \cdot R_{\text{pack}}
 \end{aligned}$$

Model B — measured curve

Direct PCHIP interpolation of a user-supplied loaded V-vs-I table; SoC is not used (the curve already reflects whatever charge state it was recorded at).

Both models clamp queries outside their data range and return a warning rather than extrapolating silently.

SoC bookkeeping. `soc_from_capacity_used(soc0, capacity_Ah, Ah_used)` = `soc0 - Ah_used/capacity_Ah`, linear discharge tracking. No temperature, C-rate-dependent-resistance, or nonlinear capacity-fade effects are modeled.

7. Motor Model

Direct drive only — the motor shaft and the propeller shaft are the same shaft ($n = n_{\text{motor}} = n_{\text{prop}}$, $Q = Q_{\text{motor}} = Q_{\text{prop}}$). No gearbox is modeled.

$$\begin{aligned}
 K_t &= (60/2\pi) / K_v = 9.5493 / K_v \quad [\text{N}\cdot\text{m}/\text{A}] \\
 Q &= K_t \cdot (I - I_0)
 \end{aligned}$$

$$V_{\text{back-emf}}(n) = 60 \cdot n / K_v \quad [n \text{ in rev/s}]$$

$$\eta_{\text{motor}} = Q \cdot \omega / (V \cdot I), \quad \omega = 2\pi n$$

8. Propeller Model

$$J = V / (n \cdot D)$$

$$T = C_T(J) \cdot \rho \cdot n^2 \cdot D^4 \quad [\text{N}]$$

$$P = C_P(J) \cdot \rho \cdot n^3 \cdot D^5 \quad [\text{W}]$$

$$Q = C_P(J) \cdot \rho \cdot n^2 \cdot D^5 / (2\pi) \quad [\text{N} \cdot \text{m}]$$

Interpolation: $C_T(J)$ and $C_P(J)$ come from tabulated test data (APC `.dat`, UIUC, or CSV), interpolated with `scipy.interpolate.PchipInterpolator` — a monotone cubic that cannot overshoot between data points. Below the table's J range the lowest table value is held (flagged `below_table_range`). At and beyond the zero-thrust crossing point, C_T is clamped to zero and C_P is held at its value at that crossing (flagged `beyond_zero_thrust`) — the interpolator can never be asked to produce negative power.

PCHIP is monotonicity-preserving, not resolution-inventing. If a source table has two consecutive J rows with the identical (rounded) C_P , PCHIP is mathematically required to be exactly flat between them — it computes zero derivative at both knots to avoid overshoot, and a Hermite cubic with equal endpoints and zero end-slopes is the constant function. This is identical to what linear interpolation would give in that same interval. A flat run of solved $n(V)/I(V)$ across a narrow J range is therefore evidence of a coarse/rounded source table (APC files publish C_P to 4 decimals), not a solver defect.

Parametric fallback (no tabulated data): linear C_T decay from a static value to zero at $J = 0.9 \cdot (\text{pitch}/D)$, constant C_P . Any candidate using it is flagged `low_confidence_parametric` throughout the UI and results.

Import validation (`data/loaders.py`): APC files are cross-checked by recomputing thrust from the parsed C_T at $\rho = 1.225 \text{ kg/m}^3$ and comparing to the file's own thrust column; a $>2\%$ mean mismatch rejects the import. Multiple RPM blocks in one file are checked for $C_T(J)$ agreement ($\pm 5\%$); if they disagree beyond that (a Reynolds-number effect at low RPM), the loader uses the single block nearest an expected operating RPM rather than blending mismatched-Reynolds data.

9. Coupled Operating-Point Solver

Source: `core/operating_point.py` . Solved as a 1-D root find in propeller speed n , not a 2-D `fsolve` over (n, I) — the torque/current relationship is algebraically substituted out first, which is both faster and immune to the false/nonsense convergence a naive 2-D solve is prone to.

For a trial n at a given V :

```
1. J      = V / (n·D)
2. (C_T,C_P) = interpolate(J)
3. Q_prop = C_P·ρ·n²·D⁵ / (2π)
4. I      = I₀ + Q_prop / K_t          [per motor]
5. V_batt = battery.terminal_voltage(motor_count·I, SoC)
6. residual(n) = V_batt - I·(R_motor+R_esc) - 60·n/K_v
```

Root-found with `scipy.optimize.brentq` over $n \in [n_{\min} \approx 10^{-5}, n_{\text{no-load}}]$, where $n_{\text{no-load}} = K_v \cdot V_{\text{batt}}(I=0) / 60$. The bracket is sign-checked *before* calling `brentq` ; if there is no sign change, the candidate returns a structured `OperatingPointFailure` with a diagnostic reason (insufficient voltage, mismatched Kv/prop, excess resistance, etc.) — never a bare exception, never a silently-wrong number.

V_batt(I) here is the loaded terminal voltage, which for the internal-resistance battery model already nets out $I \cdot R_{\text{internal}}$ internally. $R_{\text{motor}} + R_{\text{esc}}$ is added on top of that (not R_{internal} a second time) — this is what keeps the residual equivalent to " $V_{\text{ocv}} - I \cdot (R_{\text{motor}} + R_{\text{esc}} + R_{\text{internal}}) = \text{back-emf}$ " without double-counting the battery's own resistive drop, and it generalizes correctly to the measured-curve battery model, which has no separate R to extract.

Once converged: T and P_{shaft} are computed from the coefficients and aggregated by motor count (see §10); $\eta_{\text{motor}} = Q \cdot \omega / (V \cdot I)$; $\eta_{\text{prop}} = T \cdot V / P_{\text{shaft}}$ (defined as 0 at $V=0$, since useful propulsive power is zero at zero airspeed regardless of thrust).

10. Multi-Motor Extension

`motor_count` identical motor+prop+ESC units are assumed to share one pack and split the load symmetrically — each solves to the *same* per-motor operating point (self-consistent, since the motors are identical and see the same bus voltage), and the pack sees `motor_count` times the per-motor current.

Quantity	Scaling
Current (I)	per-motor; solved directly from motor circuit equation
Pack current (I_{pack})	= motor_count × I — this is what drives battery voltage sag and SoC depletion

Thrust, P_{shaft} , P_{elec}	aggregate = motor_count × per-motor value
η_{motor} , η_{prop}	per-motor ratios, unaffected by count (numerator and denominator scale identically)

Why per-motor current filters and pack current filters are separate. ESC/motor continuous-current ratings are per-unit limits; a battery C-rate limit is a pack-level limit on the sum of every motor's current. Conflating them (as a single-motor tool naturally can, since the two coincide when motor_count=1) silently misrepresents utilization once motor_count>1 — propselect reports `current_per_motor` and `current_pack` as two distinct filters for exactly this reason.

11. Ground-Roll Integration

$$\begin{aligned}
 F_{\text{net}}(V) &= T(V) - D(V) - \mu \cdot (m \cdot g - L(V)) \\
 D(V) &= 0.5 \cdot \rho \cdot V^2 \cdot S \cdot C_{D,\text{ground}} \\
 L(V) &= 0.5 \cdot \rho \cdot V^2 \cdot S \cdot C_{L,\text{ground}} \\
 ds &= m \cdot V \cdot dV / F_{\text{net}}(V) \\
 dt &= m \cdot dV / F_{\text{net}}(V)
 \end{aligned}$$

Integrated with the trapezoidal rule on a fixed dV grid (default 0.1 m/s). If $F_{\text{net}}(V) \leq 0$ anywhere, distance is reported as infinite, along with the airspeed where it went non-positive and the force deficit at that point — a diagnostic, not just a failed number.

Optional ground-effect correction

$$\begin{aligned}
 \Phi &= (16h/b)^2 / (1 + (16h/b)^2) \\
 C_{D,\text{ground}} &= C_{D0} + \Phi \cdot C_{L^2} / (\pi \cdot AR \cdot e) \quad [AR = b^2/S, \text{ derived}]
 \end{aligned}$$

Off by default. When enabled, C_{D0} (the user's "ground-roll C_D " input) is treated as the parasite baseline and the induced term above is added on top.

SoC feedback (two-pass fixed point)

Pack voltage sag depends on SoC, which depends on cumulative current draw, which depends on the (still unknown) time-of-flight, which depends on thrust, which for the internal-resistance battery model depends on SoC — circular. Solved with two passes: the first assumes constant initial SoC to get an approximate current/time profile; the second re-solves every operating point against the SoC sag implied by that profile. SoC sag over one takeoff run is small enough that this converges to a good approximation without an inner iterative solve per grid point.

12. Cruise (Steady Level Flight)

Source: `core/cruise.py`. Cruise is a single equilibrium condition, not an integration: thrust must equal drag at one airspeed, with the aircraft in steady, unaccelerated, level flight ($L=W$). There is no time-of-flight to integrate and no diagnostic sweep grid — one candidate at one cruise velocity produces exactly one operating point.

$$\begin{aligned}C_L &= 2 \cdot m \cdot g / (\rho \cdot V^2 \cdot S) \\C_D &= C_{D0, \text{cruise}} + C_L^2 / (\pi \cdot AR \cdot e) \\T_{\text{req}} &= D = 0.5 \cdot \rho \cdot V^2 \cdot S \cdot C_D\end{aligned}$$

$C_{D0, \text{cruise}}$ is a separate input from the ground-roll C_{D0} (§11) — the ground-roll value describes a gear-down, ground-proximity configuration and does not represent a clean-configuration cruise condition. Mass, wing area, span, and Oswald efficiency are reused directly from the same `AircraftConfig` as takeoff (§11); the ground-only fields (μ , $C_{L, \text{ground}}$, wheel height, ground effect) are simply unused here.

Thrust-matched propeller speed

The coupled operating-point solver (§9) has no throttle knob — it solves the single full-throttle (WOT) condition. Cruise reuses it for two purposes: as the search bound and as the feasibility ceiling.

1. $\text{WOT} = \text{solve_operating_point}(V, \dots)$ [§9, full throttle]
2. if $\text{WOT.thrust} < T_{\text{req}}$: infeasible [cannot sustain this V]
3. find $n \in [n_{\text{min}}, \text{WOT}.n]$ such that:
 $\text{motor_count} \cdot C_T(J(n)) \cdot \rho \cdot n^2 \cdot D^4 = T_{\text{req}}$ [brentq root-find, $T(n)$ at fixed V]

This works because thrust depends only on n and V , not on throttle setting, while the motor's torque/current relation $Q=K_t \cdot (I-I_0)$ (§7) holds at *any* throttle — so once n is thrust-matched, current follows directly with no throttle model needed at all:

$$\begin{aligned}Q_{\text{prop}} &= C_P(J) \cdot \rho \cdot n^2 \cdot D^5 / (2\pi) \\I &= I_0 + Q_{\text{prop}} / K_t \quad \text{[per motor, throttle-independent]} \\V_{\text{required}} &= I \cdot (R_{\text{motor}} + R_{\text{esc}}) + 60 \cdot n / K_v \quad \text{[motor terminal voltage needed]} \\\text{throttle} &= V_{\text{required}} / V_{\text{batt}}(I_{\text{pack}}, \text{SoC})\end{aligned}$$

Throttle model: linear/resistive ESC, not a switching buck converter. Partial throttle is modeled as the ESC dropping the excess voltage ($V_{\text{batt}} - V_{\text{required}}$) resistively, so pack current equals motor current exactly as at WOT ($I_{\text{pack}} = \text{motor_count} \times I$, no step-down). This is a conservative (pessimistic) assumption for partial-throttle battery draw and endurance — a real switching ESC draws less pack current than motor current at partial throttle ($I_{\text{pack}} \approx$

duty $\times I_{\text{motor}}$ for an ideal lossless buck). Upgrade path if endurance estimates need to be tighter: model duty = $V_{\text{required}}/V_{\text{batt}}$ explicitly and scale I_{pack} by it.

Endurance and range

$$\begin{aligned}\text{endurance} &= \text{SoC}_0 \cdot \text{capacity_Ah} / I_{\text{pack}} \cdot 3600 & [\text{s}] \\ \text{range} &= \text{endurance} \cdot V_{\text{cruise}} & [\text{m}]\end{aligned}$$

Endurance is reported only when the battery model exposes a finite `capacity_ah` (the measured-curve model may not).

Cruise envelope (for plotting)

`cruise_envelope()` re-solves the equilibrium above at each airspeed in a caller-supplied list — it is a thin wrapper (`dataclasses.replace` on `v_cruise_m_s` , no new physics) used by the GUI's Cruise Detail tab to plot how thrust, throttle, current, and efficiency vary across a speed range around the configured cruise velocity, since a single candidate otherwise yields only one point.

13. Closed-Form Quick Estimate

A single-point, constant-acceleration estimate shown live on the Aircraft & Requirement tab — not a substitute for the full integration, but immediately shows which term (inertia, drag, friction) dominates the requirement as inputs change.

$$\begin{aligned}V_{\text{eff}} &= V_T / \sqrt{2} \\ \bar{q} &= 0.5 \cdot \rho \cdot V_{\text{eff}}^2 \\ T_{\text{req}} &\approx \underbrace{m \cdot V_T^2 / (2s)}_{\text{(inertia)}} + \underbrace{\bar{q} \cdot S \cdot C_D}_{\text{(drag)}} + \underbrace{\mu \cdot (m \cdot g - \bar{q} \cdot S \cdot C_L)}_{\text{(friction)}}\end{aligned}$$

14. Candidate Filters

Every candidate is scored against every filter for its mode and reported in full — failures are never hard-dropped, so a combination that misses one filter by a hair stays visible next to one that misses badly.

Takeoff filters (9, source: `core/candidate.py`)

Filter	Check	Threshold source
<code>tip_mach</code>	$M_{\text{tip}} = \sqrt{((\pi n D)^2 + V^2)/a} \leq \text{limit}$, max over 0–1.2 V_T	user, default 0.75
<code>current_per_motor</code>	max per-motor $I \leq \text{motor } I_{\text{max,cont}}$	motor library
<code>current_esc</code>	max per-motor $I \leq \text{ESC } I_{\text{cont}}$ (separate filter — see §10 callout on why these aren't collapsed with min())	ESC field
<code>current_pack</code>	max pack $I \leq \text{C-rate} \times \text{capacity}$	battery C-rate field
<code>power</code>	max total $P_{\text{elec}} \leq \text{limit}$	user, optional (unlimited by default)
<code>pitch_speed</code>	informational: V_{pitch}/V_T at V_T , $V_{\text{pitch}} = n \cdot \text{pitch}$	— (no pass/fail)
<code>motor_efficiency</code>	η_{motor} at $V_T \geq \text{threshold}$	user, default 0.75
<code>prop_efficiency</code>	informational: peak η_{prop} and η_{prop} at V_T	— (no pass/fail)
<code>distance</code>	ground-roll $s \leq \text{allowed distance}$, margin % reported	user (Tab 1)

Cruise filters (9, source: `core/cruise.py`)

Same `FilterResult` framework and hard/soft convention as takeoff (§14 above); the per-motor/pack current split (§10) and current/power filters carry over unchanged. Two are new: `cruise_solve` reports whether an equilibrium exists at all, and `endurance` is cruise's hard "mission" filter, playing the same role `distance` plays for takeoff.

Filter	Check	Threshold source
<code>cruise_solve</code>	an equilibrium operating point exists at V_{cruise} (thrust-matched n found, WOT thrust $\geq$ required)	— (structural)
<code>tip_mach</code>	M_{tip} at the solved cruise $n \leq \text{limit}$	user, default 0.75
<code>current_per_motor</code>	cruise $I \leq \text{motor } I_{\text{max,cont}}$	motor library
<code>current_esc</code>	cruise $I \leq \text{ESC } I_{\text{cont}}$ (separate filter, same split rationale as §10)	ESC field

current_pack	cruise $I_{\text{pack}} \leq \text{C-rate} \times \text{capacity}$	battery C-rate field
power	battery-side P_{elec} at cruise $\leq \text{limit}$	user, optional
endurance	endurance $\geq \text{required}$ (§12)	user, optional (unlimited by default)
motor_efficiency	η_{motor} at cruise $\geq \text{threshold}$	user, default 0.75
prop_efficiency	informational: η_{prop} at cruise	— (no pass/fail)

15. Momentum-Theory Cross-Check

$$T_{\text{static}} = FM \cdot (2 \cdot \rho \cdot A)^{1/3} \cdot P_{\text{shaft}}^{2/3}, \quad FM \in [0.4, 0.75]$$

Computed per-motor at the static ($V=0$) point and compared against the solved static thrust; if outside the band, a warning is attached to the candidate ("check propeller data and units") — it is a data/unit sanity check, not a hard filter, since real propellers legitimately scatter around ideal actuator-disk theory. For multi-motor candidates, aggregate thrust/power are divided back to per-motor values first: because $T \propto P^{2/3}$ is a nonlinear (not proportional) relationship, checking totals directly would bias the band by $\text{motor_count}^{1/3}$.

16. Assumptions & Limitations

Quasi-steady propulsion. Motor and propeller are assumed to reach their equilibrium operating point instantly at every airspeed step (or, for cruise, at the single solved equilibrium) — no rotational inertia, spool-up lag, or throttle transient is modeled.

No unsteady aerodynamics; only the wing sees ground effect. The optional ground-effect correction reduces wing induced drag near the runway (takeoff only); the propeller's own thrust/power coefficients are not corrected for proximity to the ground.

Rigid, flat, level runway; one rolling-friction coefficient for the entire roll (surface presets: pavement 0.02, hard turf 0.05, short grass 0.08, mown grass 0.10, tall grass 0.15). No wind, no gusts, no crosswind. (Takeoff only — cruise has no runway.)

Symmetric multi-motor. N identical units, perfectly balanced load sharing, one shared pack/bus voltage. Asymmetric installations (different motors/props per position) are not modeled. Applies identically to cruise (§12).

Battery. The internal-resistance model is linear with SoC-only voltage sag — R_{internal} is a single constant, not itself SoC- or temperature-dependent, so the model cannot show the resistance rise (and worse sag) real packs exhibit late in discharge or when cold. No thermal effects, no capacity fade. The measured-curve model captures real nonlinearity but is frozen at whatever SoC it was recorded at. *Deliberate scope choice, not an oversight:* a full electrochemical model (e.g. PyBaMM) would replace an $O(\mu\text{s})$ algebraic lookup with a per-point PDE solve, which is incompatible with sweeping dozens-to-hundreds of candidates interactively; an SoC-indexed resistance table (same `PchipInterpolator` pattern as the OCV table) is the identified upgrade path if this fidelity gap matters.

Cruise throttle model. Partial throttle is modeled as the ESC dropping excess voltage resistively rather than as a switching buck converter — pack current equals motor current at any throttle setting, not $\text{duty} \times I_{\text{motor}}$. Conservative (pessimistic) for partial-throttle endurance/range estimates (§12).

Propeller data validity. Tabulated C_T/C_P curves are assumed valid across the solved J range at whatever Reynolds number the operating point implies. The loader explicitly flags (rather than silently accepts) RPM-block disagreement from real Reynolds effects at low RPM (§8).

Interpolation resolution. PCHIP cannot invent precision the source table doesn't have — flat regions in coarse or rounded published propeller data (e.g. APC's 4-decimal C_P) appear as genuinely flat solved $n(V)/I(V)$, not solver error (§8).

Ground roll ends at V_T (rotation/target speed), not the full liftoff transition, flare, or climb-out. Cruise is evaluated only at the configured cruise velocity/altitude, not as a full mission profile (climb, descent, reserve).

No CG shift, aeroelastic effects, or propeller wake interference between multiple motors on one airframe.

17. Validation

Check	Result
Hand-calc closed-form case ($m=8\text{kg}$, $S=0.65\text{m}^2$, $V_T=15\text{m/s}$, $s=30.5\text{m}$, $\rho=1.16$)	$T_{\text{req}}=38.6\text{N}$ reproduced exactly (inertia 76%, drag 10%, friction 14%)
Momentum-theory $D^{n_{10^3}}$ scaling ($P=750\text{W}$, $\text{FM}=0.55$, $\rho=1.16$, $D=16\text{in}\rightarrow 22\text{in}$)	$30.4\text{N} \rightarrow 37.6\text{N}$, a 24% gain, matches spec
Analytical ground-roll check ($T=\text{const}$, $D=0$, $\mu=0$)	reproduces $s = mV_T^2/2T$ to $<0.1\%$
Regression fixture	real APC 10x7 propeller + real AXI 2820/10 motor, outputs locked to specific values
Cruise equilibrium check	solved cruise thrust reproduces the independently-computed C_L/C_D drag exactly; throttle-down and infeasible (thrust-exceeds-WOT) cases both return structured, never-raise results
Automated test suite	123 tests across every module in <code>core/</code> , <code>data/</code> , and <code>project.py</code>

18. Module Reference

```
propselect/  
  core/  
    atmosphere.py    ISA density, pressure, speed of sound  
    battery.py       internal-resistance & measured-curve pack models  
    motor.py         BLDC torque/back-emf relations (direct drive)  
    propeller.py      $C_T(J)/C_P(J)$  interpolation, thrust/power/torque, momentum theory  
    operating_point.py the coupled 1-D root-find solver (single airspeed, full throttle)  
    takeoff.py       ground-roll trapezoidal integration, SoC feedback, closed-form estimate  
    candidate.py     per-candidate takeoff evaluation: ground roll + diagnostic sweep + 9 filters  
    cruise.py        steady level-flight ( $T=D$ ) equilibrium solve + 9 filters + envelope sweep  
  data/  
    loaders.py       APC .dat / UIUC / CSV import with validation  
    motors.json      bundled motor library (29 motors)  
    props/           bundled APC propeller data files (72 props)  
  project.py         JSON-serializable input model (GUI/CLI shared, takeoff + cruise)  
  main.py            CLI (propselect cli --mode takeoff|cruise|both`) and GUI launcher  
(`propselect gui`)  
  gui/              PySide6 desktop application, 10 tabs:
```

	1-4	Aircraft/Requirement, Battery/ESC, Motor Library, Propeller Library	
	5-7	Sweep & Results, Detail, Comparison	(takeoff)
	8-10	Cruise, Cruise Detail, Cruise Comparison	(cruise)
tests/	123	automated tests	

Generated from the propselect source tree. Every equation above is transcribed directly from the corresponding module's implementation, not from a separate design spec, and should stay in sync with the code as the single source of truth.